
Decision Trees

Introduction to Decision Trees

1. What is a Decision Tree?

- **Definition:**

- A **supervised learning algorithm** used for both **classification** and **regression** tasks.
- A decision tree makes decisions by **recursively splitting the dataset** based on specific feature values.
- At each step, it selects a feature that best splits the data into more homogeneous groups (for classification) or predicts a target value (for regression).

- **Tree-like Structure:**

- **Root Node:** Represents the entire dataset and the first feature split.
- **Decision Nodes:** Intermediate points where the data is split based on feature values.
- **Leaf Nodes:** Represent the final output (a predicted class for classification or a value for regression).

How Decision Trees Work

1. Tree Structure

A **decision tree** is made up of three main types of nodes:

- **Root Node:**
 - The **starting point** of the decision tree, which contains the entire dataset. All subsequent decisions stem from the root node.
- **Decision Nodes:**
 - Points in the tree where the dataset is split based on the value of a specific feature. Each split creates branches leading to further decision nodes or leaf nodes.
 - These nodes represent **questions** or **decisions** about which way to split the data (e.g., "Is the age greater than 30?").
- **Leaf Nodes:**
 - **Terminal nodes** that provide the final output of the decision tree. In a classification task, leaf nodes contain class labels, and in regression tasks, they hold predicted values.
 - Once data reaches a leaf node, no further splitting occurs, and the decision process ends.

2. Splitting the Data

At each **decision node**, the tree splits the data based on a specific feature. The main objective of each split is to create **subsets of the data** that are as **homogeneous** as possible with respect to the target variable (i.e., the class label in classification tasks or the predicted value in regression).

- The splitting process is **recursive**, meaning the tree continues to split the data until a stopping criterion is met.
- For example, in a classification task predicting whether someone will buy a car, the first decision node might split the data based on **income** (e.g., "Is the income greater than \$50,000?"). The resulting subsets are then split further using features like age or credit score.

3. Choosing Splitting Criteria

To split the data at each node, decision trees use specific criteria depending on whether they are solving a classification or regression problem.

For Classification: **Gini Impurity:**

Gini impurity measures the **probability of misclassifying** a randomly chosen data point if it were labeled according to the class distribution in the subset.

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

Where p_i is the proportion of data points belonging to class i .

- A Gini score of 0 means the subset is **perfectly homogeneous** (all points belong to the same class), while higher values indicate more impurity (i.e., mixed classes).

Information Gain (Entropy):

- **Entropy** measures the **uncertainty** or **disorder** in a dataset. The goal is to reduce uncertainty by splitting the data.
- The formula for entropy is:

$$Entropy = - \sum_{i=1}^n p_i \log_2(p_i)$$

Where p_i is the proportion of data points belonging to class i .

- Information Gain is calculated as the **reduction in entropy** after the data is split. The feature that provides the highest information gain (i.e., reduces uncertainty the most) is chosen for the split.

For Regression:

- **Mean Squared Error (MSE):**
 - For regression tasks, the splitting criterion aims to minimize the **Mean Squared Error** between the predicted values and actual values.

The formula for MSE is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the number of data points in the subset.

Key Concepts

- **Greedy Algorithm:**

- Decision trees use a **greedy algorithm** to construct the tree.
- This means they make the locally optimal decision (best split at each node) without considering the overall structure of the tree.
- While this method is efficient, it can lead to **overfitting** if the tree grows too deep.

- **Binary Trees:**

- Most decision trees are **binary trees**, meaning each decision node creates **two branches** by splitting the data into two subsets.
- Binary splits simplify the structure and make the tree easier to interpret.

Decision Tree Construction

Step 1: Selecting the Best Split

The first step in constructing a decision tree is to select the **best feature** to split the data at each decision node. This is done by evaluating each feature based on certain criteria like **Information Gain** or **Gini Impurity**. The feature that best separates the data (i.e., produces the most homogeneous groups) is chosen.

Step 2: Recursion

Once the best feature has been selected based on Information Gain, Gini Impurity, or MSE, the dataset is **split** into subsets based on the values of this feature. The tree then **recursively applies** the same process to each subset.

- **Recursive Process:**

1. **Split the data:** Based on the feature with the highest Information Gain or lowest Gini Impurity.
2. **Recompute** the splitting criteria (Information Gain, Gini, or MSE) for the new subsets.
3. **Continue splitting** until a stopping criterion is met.

For example, in a classification task, if the feature "Income" has the highest information gain, the data might be split into two groups: **Income > \$50,000** and **Income ≤ \$50,000**. Then, for each group, the process is repeated using the remaining features (e.g., Age, Credit Score).

Stopping Criteria

To avoid excessive growth and overfitting, the construction of a decision tree is controlled by **stopping criteria**:

1. **All records at a node belong to the same class:**
 - If all data points at a node are from the same class, the node becomes a **leaf node**, and no further splits are made.
2. **Maximum Tree Depth:**
 - A predefined **maximum depth** can be set to limit the growth of the tree.
 - Once the tree reaches this depth, no further splits are made, even if subsets are still heterogeneous.
3. **Minimum Samples per Leaf:**
 - A node is not allowed to split if the number of samples in a node is below a certain threshold (e.g., less than 5 samples).
 - This prevents nodes with very few samples from splitting further, which helps reduce overfitting.

Evaluating a Decision Tree

1. Classification Evaluation Metrics

- Accuracy
- Precision, Recall, and F1-Score
- Confusion Matrix

2. Regression Evaluation Metrics

- **Mean Squared Error (MSE)**: The average of the squared differences between the actual and predicted values.
 - **Usage**: Penalizes larger errors more than smaller errors.
- **Mean Absolute Error (MAE)**: The average of the absolute differences between actual and predicted values.
 - **Usage**: Less sensitive to outliers than MSE.

3. Cross-Validation

- **k-Fold Cross-Validation**: A technique to evaluate model performance by splitting the data into k subsets (folds). The model is trained on $k-1$ folds and tested on the remaining fold, repeating the process k times.
 - **Importance**: Helps assess the model's ability to generalize to new data and prevent **overfitting** by using different portions of the dataset for training and validation.
 - **Why it Matters**: Ensures that the decision tree's performance is **not overly tuned** to one subset of the data, making the model more robust and reliable.

Pruning Techniques in Decision Trees

Why Pruning is Necessary

Overfitting:

- **Overfitting** occurs when a decision tree becomes too complex by fitting not only the **true patterns** in the data but also the **noise**. This happens when the tree is allowed to grow too deep, creating many branches and leaf nodes that reflect nuances in the training data but do not generalize well to new, unseen data.
- Overfitting leads to poor performance on **test data** or real-world scenarios, where the model fails to correctly classify or predict because it has become too specific to the training data.

How Pruning Helps:

- **Pruning** is the process of **simplifying the decision tree** by removing branches and leaf nodes that contribute little to improving the model's accuracy. By doing this, pruning reduces the tree's complexity, allowing it to better generalize to unseen data and thus reduce overfitting.

A. Pre-Pruning (Early Stopping)

- **What is Pre-Pruning?**

- Pre-pruning (also known as **early stopping**) involves **stopping the growth of the tree early**, before it becomes too complex.
- This prevents the tree from overfitting by limiting how deep it can grow or by controlling other factors such as the number of samples in a node.

- **How it Works:**

- **Stopping criteria** are set before training begins. These criteria determine when to stop the tree from growing further. Pre-pruning decisions are made while building the tree, based on:
 - **Maximum Depth:** Limits the number of levels the tree can grow. For example, if a tree's depth is restricted to 3, no node can split beyond the third level.
 - **Minimum Samples Required to Split:** Limits splitting based on the number of samples available in a node. For example, you might prevent further splits if a node contains fewer than 5 samples.
 - **Minimum Samples per Leaf:** Requires each leaf node to contain a minimum number of samples, ensuring that leaf nodes are not too small.

- **Advantages:**

- **Efficiency:** The tree-building process is faster since it avoids unnecessary splits.
- **Overfitting Prevention:** By restricting the size of the tree, pre-pruning reduces the chances of capturing noise in the data.

- **Disadvantages:**

- **Risk of Underfitting:** Since pre-pruning halts tree growth before exploring all splits, it may stop too early and miss important patterns in the data.

B. Post-Pruning

- **What is Post-Pruning?**

- Post-pruning involves **growing the decision tree to its full depth** and then **removing branches** that do not improve the model's performance. Unlike pre-pruning, the entire tree is first constructed, and pruning is done after the full tree is built.

- **How it Works:**

- The tree is initially allowed to grow to its maximum possible size, capturing all patterns and even noise in the data.
- Afterward, branches and leaf nodes that contribute little to improving the model's performance are pruned. The goal is to **prune back nodes that don't add value.**
- **Evaluation Metrics:** Post-pruning typically uses **cross-validation** or a **validation set** to evaluate whether removing certain branches leads to better model performance.
- **Process:**
 1. Evaluate the tree's performance (e.g., using accuracy, MSE, or Gini).
 2. Remove the least significant branches that lead to minimal improvement.
 3. Re-evaluate the tree and keep pruning until the model's performance on the validation set starts to degrade.

- **Advantages:**

- **More Accurate:** Post-pruning allows the tree to fully explore all splits before deciding what to remove, reducing the risk of missing important splits.
- **Better Generalization:** By pruning unnecessary branches, the tree becomes more general and less likely to overfit.

- **Disadvantages:**

- **Complexity:** Post-pruning is computationally more expensive because it requires building the full tree and then testing different pruned versions to see which performs best.
- **Time-Consuming:** Pruning each branch and re-evaluating the tree can be time-intensive, especially for large datasets.

C. Cost Complexity Pruning (CCP)

- **What is Cost Complexity Pruning?**

- **Cost Complexity Pruning (CCP)** is a specific post-pruning technique that balances the trade-off between the **complexity** of the tree and its ability to fit the data. CCP adds a **penalty** for having too many branches, encouraging simpler trees that generalize better.

- **How it Works:**

- CCP minimizes the following cost function:

$$R_\alpha(T) = R(T) + \alpha|T|$$

α = penalty parameter
 $|T|$ = no. of leaf nodes
error rate

Where:

- $R(T)$ is the **error rate** of the tree,
- $|T|$ is the number of leaf nodes (a measure of tree complexity),
- α is a **penalty parameter** that controls how much weight is given to the complexity of the tree.

As α increases, the algorithm prunes more aggressively, reducing the size of the tree.

The algorithm prunes branches that **minimally reduce the training error** but add significant complexity to the tree.

- **Advantages:**

- **Balances complexity and performance:** CCP ensures the tree is not too deep, by controlling the trade-off between how well the tree fits the data and its complexity.
- **Flexibility:** The tuning of the α parameter allows for flexible control over the level of pruning, making the tree simpler without sacrificing too much accuracy.

- **Disadvantages:**

- **Choosing the right α :** Determining the optimal penalty α requires careful cross-validation, which can be computationally expensive.

Advantages of Decision Trees

a. Easy to Understand and Interpret: Decision trees are one of the most intuitive and interpretable machine learning algorithms. They represent decision-making in a **graphical, flowchart-like structure**, making it easy for non-technical stakeholders to understand how the model reaches its decisions.

Example: In a medical diagnosis system, a decision tree might classify patients based on symptoms, with each decision (e.g., "Is age > 50?") clearly laid out, making the model highly interpretable.

b. No Need for Feature Scaling: Unlike algorithms like k-NN or SVM, decision trees are **unaffected by the scale** of features because they rely on **splitting the data** based on feature values, not on distances between data points.

Example: A feature like  "income" can be measured in thousands, while "age" is measured in years, and this will not affect the tree's decision-making.

c. Handles Both Numerical and Categorical Data: Decision trees can naturally handle **both continuous (numerical) and discrete (categorical)** data types without the need for one-hot encoding or standardization. The algorithm treats categorical values just as it would continuous values by splitting based on the distinct categories.

Example: A decision tree can easily split on a categorical variable like "City" or a numerical variable like "Age" within the same model.

d. Non-parametric: Decision trees are **non-parametric**, meaning they make no assumptions about the underlying data distribution. This flexibility allows them to perform well on a variety of datasets, including those that do not follow standard assumptions like linearity or normal distribution.

Example: Whether your data is normally distributed or highly skewed, decision trees can still be applied without needing to transform the data.

Limitations of Decision Trees

a. **Overfitting**: One of the most common issues with decision trees is **overfitting**. As the tree grows deeper, it starts capturing **noise and irregularities** in the training data, making it fit too closely to the training data and perform poorly on unseen data.

Example: If a tree is allowed to split endlessly, it may reach pure leaf nodes with just one or two data points, effectively memorizing the training set. This makes the tree highly specific to the training data, reducing its generalization ability.

b. **Instability**: Decision trees can be highly **unstable** because small changes in the training data (e.g., removing or adding a few samples) can lead to a **completely different tree structure**. This is because the algorithm is greedy, choosing the best split at each step, which may not lead to the globally optimal solution.

Example: A small perturbation in data, such as a single feature value changing slightly, can cause a different tree structure and entirely different predictions, making the model sensitive to small variations.

c. **Bias Toward Splits on Categorical Variables**: Decision trees tend to **favor splits** on categorical variables with many levels (e.g., zip codes), which can introduce **bias**. The algorithm may choose such features for a split because they inherently create more branches, even if they are not the most informative feature.

Example: A tree might split on a variable like "Zip Code" just because it has many categories, even though this split might not be as predictive as a simpler feature like "Age" or "Income."

d. **Limited Expressiveness**: Decision trees create **orthogonal decision boundaries** because they split the data along feature axes. This can be limiting in cases where the true decision boundary between classes is more complex or non-linear.

Example: If the relationship between features is curved or diagonal, a decision tree may struggle to capture this, as it can only split data vertically or horizontally, potentially missing more complex patterns.

Applications of Decision Trees

Decision trees are widely used across multiple industries due to their versatility. Some prominent applications include:

1

a. Medical Diagnosis

- **Use Case:** Decision trees help classify patients based on symptoms and medical test results to predict diseases or health conditions.
- **Example:** Classifying whether a patient has a heart disease based on features like age, cholesterol level, and blood pressure. The decision tree makes predictions by splitting the data based on these symptoms and lab results.

2

b. Customer Segmentation

- **Use Case:** Businesses use decision trees to identify different types of customers based on their purchasing behavior and demographic information.
- **Example:** A marketing company might use a decision tree to segment customers based on income, age, and spending patterns. This helps in targeting specific products to relevant customer groups.

3

c. Fraud Detection

- **Use Case:** Decision trees are used to detect fraudulent transactions based on transaction attributes such as transaction amount, location, and frequency.
- **Example:** A decision tree may flag unusual credit card transactions (based on location, time, or amount) as potentially fraudulent by identifying patterns in past fraudulent transactions.

Probabilistic Reasoning

Motivation: Why Do We Need Probabilistic Reasoning?

1. Dealing with Uncertainty

Most real-world situations involve some level of uncertainty. For example:

- In **medical diagnosis**, a doctor might not know for sure if a patient has a disease based on symptoms alone.
- In **weather prediction**, various factors affect the outcome, but none of them can predict the future with 100% certainty.

Probabilistic reasoning helps AI systems model this uncertainty by representing knowledge in terms of probabilities, rather than absolute truths. This allows AI to make more flexible and reliable decisions.

2. Handling Incomplete or Noisy Data

AI often operates with incomplete or noisy data. Probabilistic models, like Bayesian networks or Markov models, allow AI systems to infer the likelihood of certain events even when they don't have full information.

- **Example:** In self-driving cars, sensor data might be incomplete due to bad weather or occlusions. Probabilistic reasoning helps the car infer what's likely happening around it, despite imperfect data.

3. Learning from Data

Probabilistic models, such as those used in machine learning, allow AI systems to learn from data. Rather than fitting a deterministic model, probabilistic models estimate distributions of outcomes and update their beliefs as new data becomes available.

- **Example:** In **speech recognition**, probabilistic reasoning helps estimate which word was most likely spoken based on noisy audio signals.

4. Decision Making Under Uncertainty

AI systems frequently have to make decisions where the outcomes are uncertain. Probabilistic reasoning enables AI to weigh the likelihood of different outcomes and choose the best course of action.

- **Example:** In **autonomous robots**, probabilistic reasoning helps the robot decide whether to take a particular path, given that it may not have full knowledge of obstacles ahead.

5. Modeling Real-World Complexities

Real-world processes are often too complex to model deterministically. Probabilistic reasoning allows AI systems to represent these complexities by modeling **random variables** and their interactions. Bayesian Networks, for example, can capture relationships between multiple factors, even when they're uncertain.

- **Example:** In **financial markets**, probabilistic models are used to predict market movements by considering various random variables, such as stock prices, interest rates, and economic indicators.

6. Flexibility in Updating Beliefs

Probabilistic reasoning allows AI systems to update their beliefs as new evidence comes in. Bayesian reasoning, for example, is particularly useful for dynamically adjusting the probabilities of different outcomes as more data is observed.

- **Example:** In **spam filtering**, probabilistic reasoning helps the system improve its accuracy as more emails are classified and new patterns are learned.

7. Robustness in Complex Systems

In multi-component systems where various parts interact in uncertain ways, probabilistic reasoning helps maintain robustness by factoring in uncertainty about each part of the system. This is critical for building reliable AI systems that need to function in unpredictable environments.

Examples of Uncertainty in AI

1. Medical Diagnosis

In medical AI systems, uncertainty arises because symptoms and test results do not always lead to a definitive diagnosis. For instance, a set of symptoms could correspond to multiple diseases, and test results can have false positives or false negatives. AI systems must deal with these uncertainties to provide probabilistic diagnoses.

- **Example:** An AI model predicting the likelihood of heart disease based on patient data like cholesterol levels, age, and symptoms might give a probability (e.g., 70%) rather than a definite outcome, acknowledging uncertainty due to incomplete information.

2. Self-Driving Cars

Autonomous vehicles rely on sensors like cameras, radar, and LiDAR, but these sensors are prone to noise and incomplete information, especially in poor weather conditions or heavy traffic. Uncertainty arises in detecting obstacles or predicting pedestrian behavior.

- **Example:** A self-driving car may be unsure whether an object detected by its sensors is a pedestrian or a stationary object due to fog, rain, or other environmental factors.

3. Speech Recognition

In natural language processing and speech recognition, uncertainty arises because spoken language can be ambiguous, affected by accents, background noise, or unclear pronunciations. AI systems must estimate the probability of different words being spoken.

- **Example:** A voice assistant interpreting "weather" could also hear "whether" due to phonetic similarity, especially in a noisy environment, making the system uncertain about the correct interpretation.

4. Financial Market Predictions

In AI systems used for stock market predictions, uncertainty arises due to the complexity and unpredictability of markets. Economic indicators, political events, and unforeseen circumstances can all affect the outcome in ways that are difficult to predict.

- **Example:** An AI model might predict a 60% chance that a stock will increase in value based on historical data and market trends, but the actual outcome remains uncertain due to unexpected factors like global events.

5. Weather Forecasting

Uncertainty is inherent in weather prediction models due to the chaotic nature of weather systems and the incomplete data available from satellites and ground stations. AI-powered weather forecasting models generate probabilities for various outcomes (e.g., 70% chance of rain) instead of deterministic predictions.

- **Example:** A weather prediction system might give a 40% chance of rain based on cloud patterns, temperature, and humidity, but it cannot be entirely certain due to changing atmospheric conditions.

6. Natural Language Processing (NLP)

In NLP, uncertainty arises due to the multiple possible meanings of words or phrases in different contexts. Language models must make probabilistic decisions about the most likely interpretation of text based on context.

- **Example:** When processing the sentence "He saw her duck," an NLP model must choose between different interpretations: whether "duck" refers to the action of ducking or the bird, leading to uncertainty in meaning.

7. Recommendation Systems

Recommendation systems, such as those used by Netflix or Amazon, operate under uncertainty because they don't have complete knowledge of user preferences. They make probabilistic estimates of what a user might want to watch or buy based on limited historical data.

- **Example:** A movie recommendation system might suggest a movie with 80% confidence that the user will enjoy it, but it remains uncertain because the user's preferences could change or the data could be incomplete.

8. Autonomous Robotics

In robotics, especially autonomous robots, uncertainty arises in navigation and object recognition tasks. Robots must make decisions based on noisy sensor data and incomplete maps of the environment.

- **Example:** A delivery robot navigating through a crowded street might face uncertainty about its surroundings due to dynamic obstacles like pedestrians or vehicles moving unpredictably.

9. Machine Translation

AI models for translating languages often face uncertainty because there are multiple ways to translate a sentence, and context, idiomatic expressions, or cultural nuances can lead to ambiguities in meaning.

- **Example:** A translation system might translate the phrase "He gave me a cold shoulder" literally, but it faces uncertainty because the phrase could mean the person was being unfriendly (idiomatic meaning).

10. Fraud Detection in Banking

In financial services, AI systems used for detecting fraud in transactions face uncertainty due to the subtle differences between normal and fraudulent behavior. The model often produces a probability that a transaction is fraudulent rather than a binary decision.

- **Example:** A credit card transaction in a foreign country might trigger a fraud alert with a 60% chance of being classified as fraudulent, but the system is uncertain because it lacks complete knowledge of the cardholder's travel plans.

Introduction to Probability Theory (Basic Concepts)

1. Random Variables

- A **random variable** represents an outcome of a probabilistic event.
 - Example: Let X represent the outcome of rolling a die. X can take on values $\{1,2,3,4,5,6\}$.
- **Discrete random variables** take on a countable set of values (e.g., the outcome of a dice roll), while **continuous random variables** take values from a continuous range (e.g., temperature measurements).

2. Probability Distributions

- A **probability distribution** assigns probabilities to the possible values of a random variable.
 - **For discrete variables:** $P(X=x)$ gives the probability that X takes value x .
 - **For continuous variables:** The probability is described by a **probability density function (PDF)**.

Example (Discrete):

For a fair die, the probability distribution is uniform:

$$P(X=i) = 1/6, \quad i \in \{1,2,3,4,5,6\}$$

Example (Continuous):

For a normal (Gaussian) distribution:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where μ is the mean and σ^2 is the variance.

Key Definitions in Probability Theory

1. Joint Probability Distribution: A **joint probability distribution** represents the probability of multiple random variables taking specific values at the same time.

Example:

If X is the outcome of rolling a die and Y is flipping a coin, the joint probability distribution might be represented as $P(X=2, Y=Heads)$.

2. Marginalization (Summing Over Variables): **Marginalization** is the process of summing over the possible values of one random variable to get the probability distribution of another.

Example: If we know the joint distribution $P(X, Y)$, we can marginalize Y to get the distribution for X :

$$P(X) = \sum_y P(X, Y = y)$$

This allows us to focus on one variable by ignoring the others.

3.Conditional Probability: **Conditional probability** is the probability of one event happening given that another event has occurred.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

This tells us the likelihood of event A occurring, knowing that B is true.

Example: If we know a person is coughing, the probability they have a cold could be computed as $P(\text{Cold}|\text{Cough})$.

4.Independence: Two events A and B are **independent** if the occurrence of one does not affect the probability of the other:

$$P(A \cap B) = P(A)P(B)$$

Example: Rolling two dice independently. The outcome of one die doesn't affect the other.

Bayesian Networks

What is a Bayesian Network (BN)

A **Bayesian Network** is a graphical model that represents the probabilistic relationships between a set of variables using a **Directed Acyclic Graph (DAG)**. It is used to represent **joint probability distributions** in a way that captures conditional dependencies among variables efficiently.

Key Components:

- **Nodes:** Each node in the graph represents a **random variable**. For example, in a medical diagnosis context, a node could represent the presence of a disease or a symptom.
- **Edges:** Directed edges between nodes represent **conditional dependencies** between the variables. If there is a directed edge from node X to node Y, then Y is conditionally dependent on X.

Formal Definition: A Bayesian Network is a probabilistic graphical model that represents a set of random variables $X_1, X_2, \dots, X_n$ and their conditional dependencies via a directed acyclic graph (DAG).

Example:

Consider a simple **Bayesian Network** for a medical diagnosis scenario:

- D represents a disease.
- S represents symptoms caused by the disease.

In this case, there could be a directed edge from D to S, indicating that the presence of the disease influences the likelihood of the symptom.

Structure of a Bayesian Network: Directed Acyclic Graph (DAG)

A Bayesian Network is structured as a **Directed Acyclic Graph (DAG)**, which has the following properties:

- **Directed:** Each edge has a direction, indicating a cause-and-effect relationship (e.g., $X \rightarrow Y$ means X influences Y).
- **Acyclic:** The graph has no cycles, meaning you cannot start from one node and follow a series of directed edges back to the same node. This ensures that the model represents a valid causal structure.

Conditional Independence:

In a Bayesian Network, each node is conditionally independent of its non-descendants given its parents. This allows for a compact representation of the joint probability distribution.

Example:

Consider a simple medical diagnosis Bayesian Network:

- D (Disease) causes symptoms S1 (Symptom 1) and S2 (Symptom 2).
- The DAG structure is:

$D \rightarrow S1$

$D \rightarrow S2$

- This structure implies that S1 and S2 are conditionally independent given D, i.e., knowing whether a patient has S1 provides no additional information about S2 if we already know whether the patient has the disease.

Semantics of Bayesian Networks

The semantics of a Bayesian Network lies in its ability to **factorize** the **joint probability distribution** of the random variables based on the structure of the DAG.

Joint Probability Distribution Factorization:

In a Bayesian Network, the joint probability distribution of a set of random variables $X_1, X_2, \dots, X_n$ can be factorized as:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$

- X_i is a random variable.
- $\text{Parents}(X_i)$ represents the set of variables that are the direct causes of X_i (i.e., the parent nodes of X_i in the DAG).
- Each term in this product represents the **conditional probability** of X_i given its parents.

Explanation:

- The joint probability distribution of the entire system can be represented as a product of **local conditional distributions**.
- This factorization significantly reduces the complexity of representing the joint distribution, as each variable only depends on a subset of other variables (its parents) rather than the entire system.

Example: Factorization in a Medical Diagnosis Network

Suppose we have the following Bayesian Network for a diagnosis problem:

- D (Disease) can cause:
 - S1 (Symptom 1)
 - S2 (Symptom 2)
- The structure is:

$D \rightarrow S1$

$D \rightarrow S2$

- The **joint probability distribution** of this system is: $P(D, S1, S2) = P(D) \cdot P(S1|D) \cdot P(S2|D)$

Here, the joint probability distribution is factored into three terms:

1. $P(D)$: The prior probability of having the disease.
2. $P(S1|D)$: The conditional probability of having symptom S1 given the disease.
3. $P(S2|D)$: The conditional probability of having symptom S2 given the disease.

By representing the joint probability in this way, we reduce the complexity from computing the full joint distribution of all variables to computing simpler local conditional probabilities.

Factorization Reduces Complexity

Without factorization, the **full joint probability distribution** of n binary random variables would require 2^n parameters. For large networks, this becomes infeasible.

In a Bayesian Network, the factorization based on the **conditional dependencies** between variables allows us to reduce the number of parameters significantly. Instead of needing to store all 2^n probabilities, we only need to store the conditional probabilities for each variable given its parents.

The number of parameters grows linearly with the number of edges in the network, rather than exponentially with the number of variables.

Example: Medical Diagnosis Network

Consider a medical diagnosis scenario with three variables: D: Disease; S: Symptom; T: Test result

In a full joint probability model, we would need to store $2^3 = 8$ probabilities. In a Bayesian network, where we assume that:

- S depends on D (i.e., $P(S|D)$)
- T depends on D (i.e., $P(T|D)$)
- D is independent

This factorization would only require us to store:

- $P(D)$
- $P(S|D)$
- $P(T|D)$

This requires far fewer probabilities than the full joint distribution.

Inference in Bayesian Networks: Types of Inference

1. Diagnostic Inference:

- **What is it?**

Diagnostic inference is reasoning **from effects (symptoms) to causes**.

- We aim to compute the probability of a cause given that we have observed an effect.

$$P(\text{Cause}|\text{Effect})$$

- **Example:**

In a medical diagnosis, suppose we want to compute the probability that a patient has a disease (cause) given that they exhibit certain symptoms (effect). This would involve computing $P(\text{Disease}|\text{Symptoms})$.

Mathematical Formulation:

$$P(D|S) = \frac{P(S|D)P(D)}{P(S)}$$

Here, we use **Bayes' theorem** to invert the relationship, moving from effect to cause.

2. Predictive Inference:

- **What is it?**

Predictive inference is reasoning **from causes to effects**. This is the most intuitive type of reasoning where we compute the probability of an effect given some known causes. $P(\text{Effect}|\text{Cause})$

- **Example:** For predicting symptoms based on a disease, we want to compute $P(\text{Symptoms}|\text{Disease})$. Given the disease (cause), we can predict the likelihood of observing certain symptoms.

3. Intercausal Inference:

- **What is it?**

Intercausal inference involves reasoning about the **interaction between different causes** of a single effect. When two causes affect the same effect, observing one cause can influence our belief about the other.

- **Example:** Consider two causes, D1 (Disease 1) and D2 (Disease 2), both of which can lead to the same symptom S (Fever). If we know that the symptom S has occurred, the likelihood of each disease changes, but observing one disease might reduce the likelihood of the other, since both diseases explain the same symptom.

4. Combined Inference:

- **What is it?**

Combined inference refers to a combination of diagnostic, predictive, and intercausal reasoning. This is often required in complex networks where both causes and effects are present, and we must reason both ways.

- **Example:** Suppose we observe a symptom (effect) and know something about the patient's risk factors (cause). We would combine both predictive and diagnostic inference to reason about the probabilities of various diseases.